

Especificación técnica para el piloto

Tres piezas a construir antes de abrir · con criterios de aceptación

Kairo — documento para quien lo implemente

11 de septiembre de 2026

Tres piezas, por orden de importancia: **(1)** plan de piloto con caducidad —sin esto no hay piloto—, **(2)** tope de gasto agregado, **(3)** aviso externo de incidencias bloqueantes. Estimación conjunta: 3–4 días de trabajo de quien conozca el repo, más pruebas. Ninguna de las tres toca el motor de Camino ni la corrección, así que se pueden hacer en paralelo al resto.

Pieza 1 — Plan de piloto con caducidad

El problema, en concreto

Nueve rutas de servidor cortan al alumno con `!billing.hasActivePack && billing.daysSince >= 7`, y el plan Free limita el Camino a `maxStudyDaysPerWeek: 2` y las fotos a 3 al mes. Las tres puertas dependen de lo mismo, así que las tres se abren con una sola cosa: **una fila activa en `user_entitlements`**.

`app/lib/billing/serverUsage.ts:20–31` lo deja claro — `hasActivePack` es simplemente «existe un entitlement con `status='active'` y `expires_at` en el futuro».

Diseño propuesto

Decisión de diseño importante: canje puntual, no comprobación por petición

El mecanismo actual de cortesía (`grantCourtesyAccessIfEligible`) se ejecuta *dentro* de `getUserBillingContext`, es decir, en cada petición de IA de cada alumno. Para el piloto no hace falta y añade riesgo: mejor un **endpoint de canje que se llama una vez** e inserta la fila. Después, `getUserBillingContext` la encuentra sin ningún cambio. Cero modificaciones en el camino caliente.

Tabla nueva (`pilot_codes`), siguiendo el patrón de `parent_checkout_links`:

Columna	Tipo	Notas
<code>id</code>	uuid pk	
<code>code_hash</code>	text unique	Hash del código, no el código en claro
<code>label</code>	text	«2ºA IES X», para saber quién es quién en los datos
<code>plan_id</code>	text	premium durante el piloto
<code>grants_until</code>	timestampz	Fin del acceso concedido (D14 + margen)
<code>max_redemptions</code>	integer	Tope duro, p. ej. 200
<code>redemptions</code>	integer	Contador
<code>expires_at</code>	timestampz	Cuándo deja de poder canjearse el código
<code>revoked_at</code>	timestampz null	Para poder cortar de golpe si se filtra

Endpoint POST `/api/pilot/redeem`, autenticado:

1. Valida el código contra `code_hash`; rechaza si está revocado, caducado o sin canjes.
2. Comprueba que el alumno **no tenga ya** un entitlement activo (no regalar encima de uno de pago).
3. Inserta en `user_entitlements`: `plan_id` del código, `source: 'manual_admin'`, `status: 'active'`, `expires_at = grants_until`, y en `metadata` el `label` y el `id` del código — eso es lo que luego permite filtrar el piloto en las métricas.
4. Incrementa **redemptions en la misma transacción** que el insert (función SQL, no dos llamadas: es exactamente el fallo que ya costó caro en el otorgamiento de XP).

Seguridad, sin discusión:

- Un canje por cuenta: índice único sobre (`user_id`, código) en el metadata o tabla de canjes.
- Límite de intentos por cuenta e IP (los códigos se adivinan si se dejan probar mil veces).
- `revoke all` a `public`, `anon` y `authenticated` sobre cualquier función nueva, y `grant` solo a `service_role`. Es el fallo que ya apareció en `increment_camino_progress`; no repetirlo.
- El código se genera y se entrega en clase; no se envía por correo masivo.

Criterios de aceptación

- Una cuenta nueva que canjea el código puede corregir el **día 9** (hoy no puede).
- Esa misma cuenta ve el Camino con más de 2 días por semana y puede subir más de 3 fotos.
- Al pasar `grants_until`, vuelve al comportamiento Free sin intervención manual.
- Un código revocado deja de funcionar de inmediato para canjes nuevos, y hay una forma documentada de revocar los accesos ya concedidos.
- Dos canjes simultáneos de la misma cuenta dejan **un solo** entitlement.
- El alumno del piloto es distinguible en las métricas por el `label` de su clase.

Detalle a tener en cuenta: hoy falla cerrado

`getUserBillingContext` tiene un `catch` que devuelve `hasActivePack: false` ante cualquier error (línea 37-39). Con 150 alumnos a la vez, un error transitorio de Supabase convierte a un alumno del piloto en usuario Free y le cierra la puerta de los 7 días. Durante el piloto merece la pena distinguir «no tiene acceso» de «no he podido comprobarlo» y, en el segundo caso, dejar pasar y registrar el incidente.

Pieza 2 — Tope de gasto agregado

El problema

Los límites por alumno existen (25 o 200 correcciones al mes según plan), pero **no hay ningún tope global**. Si una clase entera hace simulacros a la vez, o alguien descubre que el chat sale gratis, nada corta. El escenario realista del piloto son 135–400 USD, pero el techo teórico son ~1.350 USD y no hay nada que lo impida.

Diseño propuesto

- Un presupuesto configurable por ventana (día y piloto completo), en variable de entorno para poder cambiarlo sin desplegar.
- El acumulado sale de `ai_usage_events.estimated_cost_eur`, que ya se registra por usuario, ruta y acción.
- **Dos umbrales, no uno:** al 70 % avisa (email al equipo); al 100 % corta las llamadas nuevas y devuelve un mensaje honesto al alumno («hoy no podemos corregir más, vuelve mañana»), nunca un error genérico.
- El corte debe ser por **ventana diaria** además de total: así un pico de un día no consume el presupuesto de las dos semanas.

Criterios de aceptación

- Simulado un gasto por encima del umbral, las rutas de IA devuelven el mensaje controlado y dejan de llamar al proveedor.

- El aviso del 70 % llega de verdad a un buzón que alguien mira.
- El corte se puede levantar subiendo la variable, sin desplegar.
- Un corte queda registrado como incidencia, no solo en los logs.

Pieza 3 — Aviso externo de incidencias bloqueantes

El problema

El panel de incidencias se refresca cada minuto **solo mientras alguien lo tiene abierto**. En un piloto presencial, un fallo a las 10:15 que afecte a una clase entera se descubre cuando alguien se acuerde de mirar.

Diseño propuesto

- Al registrar una incidencia con severidad **blocking**, enviar un email (Resend ya está configurado y con dominio verificado) a la lista del equipo.
- **Agrupar por huella**: un fallo que afecta a 30 alumnos manda un aviso, no treinta. El campo **fingerprint** ya existe para eso; el aviso se manda en la primera aparición y luego, como mucho, un recordatorio cada hora mientras siga abierta.
- Incluir en el correo lo único que hace falta para decidir: código, ruta, cuántos alumnos distintos y desde cuándo. Nada de datos del alumno.

Criterios de aceptación

- Una incidencia bloqueante de prueba genera un correo en menos de un minuto.
- Treinta apariciones de la misma incidencia no generan treinta correos.
- Ninguna respuesta ni correo incluye datos personales ni contenido de las respuestas del alumno.

Lo que NO hay que hacer ahora

Fuera de alcance hasta después del piloto

- Resolver la contradicción «Free para siempre» vs. corte a los 7 días en la página de precios. Es un bloqueante para **cobrar**, no para el piloto, y tocarlo ahora mezcla dos decisiones distintas.
- Unificar los dos generadores de Camino.
- Añadir asignaturas nuevas (Biología, Latín...). No da tiempo a hacerlo bien y una asignatura mal cubierta hace más daño que una ausente y avisada.
- Rediseñar el panel de incidencias. Con que avise y agrupe, basta.

Orden y dependencias

Orden	Pieza	Bloquea a	Estimación
1	Plan de piloto	Todo el piloto	1-2 días
2	Aviso externo de incidencias	Fase 1 (semilla)	~0,5 día
3	Tope de gasto	Fase 3 (los 150)	~1 día

Las tres son independientes entre sí: se pueden repartir. La 1 es la única que bloquea la fecha de inicio.